

Week 9 Homework Assignment

List Algorithms and Search

Total Points: 100

Name: _____

Class: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see [instructions.pdf](#)
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Algorithm Pattern Recognition (12 points)

Time estimate: 15 minutes

1.1 Exercise 1.1: Identify the Pattern (12 points)

For each code snippet below, identify which algorithm pattern it uses and describe what it does.

#	Code Snippet	Pattern Name	What It Does
1	total = 0 for num in numbers: total += num	_____	_____
2	result = [] for n in nums: if n > 10: result.append(n)	_____	_____
3	for i in range(len(lst)): if lst[i] == target: return i return -1	_____	_____
4	maximum = nums[0] for n in nums: if n > maximum: maximum = n	_____	_____

Pattern choices: Accumulator, Filter, Linear Search, Min/Max Finder

2 Part 2: Linear Search Implementation (15 points)

Time estimate: 20 minutes

2.1 Exercise 2.1: Complete the Search Function (8 points)

Complete this function that finds ALL occurrences of a target value in a list:

```
# week09_ex2_1.py
# Name: _____
# Find all positions where target appears
```

```
def find_all_positions(my_list, target):  
    """  
    Returns a list of all indices where target appears in my_list.  
    If target not found, returns empty list.  
    """  
    positions = []  
    # YOUR CODE HERE: Loop through list and collect all matching indices  
  
    return positions  
  
# Test your function  
scores = [85, 92, 78, 85, 95, 85, 88]  
print(find_all_positions(scores, 85)) # Should print: [0, 3, 5]  
print(find_all_positions(scores, 100)) # Should print: []
```

Save this code as `week09_ex2_1.py`

2.2 Exercise 2.2: Search with a Condition (7 points)

Write a function that finds the FIRST item in a list that meets a condition:

```
# week09_ex2_2.py  
# Find first passing grade  
  
def find_first_passing(grades, passing_score=60):  
    """  
    Returns the first grade that is >= passing_score.  
    Returns None if no passing grades found.  
    """  
    # YOUR CODE HERE  
  
  
# Test cases  
all_grades = [45, 52, 78, 90, 55]  
print(find_first_passing(all_grades)) # Should print: 78  
print(find_first_passing(all_grades, 95)) # Should print: None
```

Save this code as `week09_ex2_2.py`

3 Part 3: Sorting Practice (13 points)

Time estimate: 15 minutes

3.1 Exercise 3.1: Sort vs Sorted - Memory Diagram (6 points)

Draw memory diagrams showing what happens with each approach:

Approach 1: Using `sort()`

```
numbers = [5, 2, 8, 1, 9]  
numbers.sort()  
print(numbers)
```

Draw memory diagram showing:
Before: numbers → [5, 2, 8, 1, 9]
After: numbers → [1, 2, 5, 8, 9] (same object, modified)

Approach 2: Using sorted()

```
numbers = [5, 2, 8, 1, 9]
sorted_nums = sorted(numbers)
print(numbers)
print(sorted_nums)
```

Draw memory diagram showing:
numbers → [5, 2, 8, 1, 9] (unchanged)
sorted_nums → [1, 2, 5, 8, 9] (new object)

3.2 Exercise 3.2: When to Use Which? (7 points)

For each scenario, circle whether you should use `sort()` or `sorted()`, and explain why:

Scenario 1: You need to display student names alphabetically, but also need to keep the original order for later.

Use: `sort()` / `sorted()`

Reason: _____

Scenario 2: You have a list of high scores that you want to permanently arrange from highest to lowest.

Use: `sort()` / `sorted()`

Reason: _____

Scenario 3: You're comparing two sorting algorithms and need to sort the same list twice with different methods.

Use: `sort()` / `sorted()`

Reason: _____

4 Part 4: List Comprehensions (15 points)

Time estimate: 20 minutes

4.1 Exercise 4.1: Convert Loops to Comprehensions (9 points)

Rewrite each loop as a list comprehension:

Example A:

```
# Original loop
result = []
for x in range(10):
    result.append(x * 2)
```

List comprehension: `result =` _____

Example B:

```
# Original loop
passing = []
for grade in all_grades:
    if grade >= 60:
        passing.append(grade)
```

List comprehension: `passing =` _____

Example C:

```
# Original loop
lengths = []
for word in words:
    lengths.append(len(word))
```

List comprehension: `lengths =` _____

4.2 Exercise 4.2: Write New Comprehensions (6 points)

Write list comprehensions for these tasks:

Task 1: Create a list of squares for numbers 1 through 10.

Task 2: From a list of names, create a list containing only names longer than 5 characters.

Task 3: Given `prices = [100, 250, 80, 150]`, create a list with a 20% discount applied to each price.

5 Part 5: Algorithm Design (15 points)

Time estimate: 25 minutes

5.1 Exercise 5.1: Plan Your Algorithm (15 points)

You need to write a function that finds the position of the smallest even number in a list.

Step 1: Write Pseudocode (6 points)

1. Initialize variables to track smallest even number and its position
2. _____
3. _____
4. _____
5. _____
6. Return the position (or -1 if no even numbers found)

Step 2: Implement Your Algorithm (9 points)

```
# week09_ex5_1.py
# Find position of smallest even number

def find_smallest_even_position(numbers):
    """
```

```
    Returns the index of the smallest even number.
    Returns -1 if no even numbers in list.
    """
    # YOUR CODE HERE - implement your pseudocode


# Test cases
print(find_smallest_even_position([3, 8, 5, 2, 9, 4])) # Should return 3 (
    value 2 at index 3)
print(find_smallest_even_position([1, 3, 5, 7]))        # Should return -1 (
    no evens)
print(find_smallest_even_position([10, 2, 6, 2, 8]))    # Should return 1 (
    first occurrence of 2)
```

Save this code as `week09_ex5_1.py`

6 Part 6: Debugging Challenge (12 points)

Time estimate: 20 minutes

Each program has bugs related to list algorithms. Find ALL errors and write the corrected version.

6.1 Buggy Code 1: Search Function (6 points)

```
# This function should find if a name exists in a list
# It has 2 errors - find and fix them!

def search_name(names, target):
    for i in range(len(names)):
        if names[i] = target:
            return True
    return False

students = ["Fatima", "Ali", "Zara", "Hassan"]
print(search_name(students, "Zara"))
```

Error 1: _____

Error 2: _____

Fixed Code:

```
# Write the corrected version here:
```

6.2 Buggy Code 2: Sorting Gone Wrong (6 points)

```
# This should sort temperatures and find the lowest
# But something's wrong - what is it?

temperatures = [32, 28, 35, 30, 29]
lowest = sorted(temperatures)
print(f"Lowest temperature: {lowest}")
```

What's wrong? _____

Fixed Code:

```
# Write the corrected version:
```

Save your corrected code as `week09_ex6_fixed.py`

7 Part 7: Code Tracing (8 points)

Time estimate: 10 minutes

7.1 Exercise 7.1: Trace This Search (8 points)

Trace through this linear search step by step:

```
def find_index(items, target):  
    for i in range(len(items)):  
        if items[i] == target:  
            return i  
    return -1  
  
result = find_index([10, 20, 30, 40], 30)
```

Fill in the trace table:

Step	i	items[i]	Comparison	Action
1	0		10 == 30?	
2				
3				
4				

Final return value: _____

8 Part 8: Full Program Development (20 points)

Time estimate: 35 minutes

8.1 Exercise 8.1: Cricket Score Analyzer (20 points)

Create a complete program that analyzes cricket scores using list algorithms.

Program Requirements

Your program should:

- Store cricket scores for multiple players in a list
- Use linear search to find a specific score
- Sort scores to find top performers
- Use list comprehensions to filter high scores (50+)
- Display organized results with proper formatting
- Handle edge cases (empty list, no high scores)

Example Run:

=== Cricket Score Analyzer ===

Enter player scores (comma-separated): 45, 78, 23, 92, 67, 34, 88

Analysis Results:

Total players: 7

Scores in order: [23, 34, 45, 67, 78, 88, 92]

Highest score: 92

Lowest score: 23

High scores (50+): [78, 92, 67, 88]

Search for a score: 67

Found! Score 67 is at position 4 in the original list.

Search for a score: 100

Score 100 not found in the list.

Starter Structure:

```
# cricket_analyzer.py
# Name: -----
# Date: -----
# Cricket Score Analyzer using List Algorithms

def get_scores():
    """Get scores from user and return as a list of integers."""
    # YOUR CODE HERE
    pass

def find_score(scores, target):
    """Search for a score and return its position (or -1)."""
    # YOUR CODE HERE - use linear search
    pass

def analyze_scores(scores):
    """Analyze and display score statistics."""
    # YOUR CODE HERE
    # Use sorting to find highest/lowest
    # Use list comprehension for high scores (50+)
    pass

def main():
    print("=== Cricket Score Analyzer ===")

    # Get input
    scores = get_scores()

    # Check if empty
    if not scores:
        print("No scores entered!")
        return

    # Display analysis
    analyze_scores(scores)

    # Interactive search
    while True:
        search_input = input("\nSearch for a score (or 'quit' to exit): ")
        if search_input.lower() == 'quit':
            break
```

```
try:
    target = int(search_input)
    position = find_score(scores, target)

    if position != -1:
        print(f"Found! Score {target} is at position {position}.")
    else:
        print(f"Score {target} not found in the list.")
except ValueError:
    print("Please enter a valid number!")

# Run the program
if __name__ == "__main__":
    main()
```

Testing Checklist:

- ☐ Program runs without errors
- ☐ Handles empty input gracefully
- ☐ Linear search returns correct positions
- ☐ Sorting works correctly (use `sorted()` to preserve original)
- ☐ List comprehension filters high scores correctly
- ☐ Output is formatted clearly
- ☐ Search feature works for both found and not-found cases

Save this program as `cricket_analyzer.py`

9 Part 9: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. **Algorithm Patterns:** Which algorithm pattern (accumulator, filter, search, min/max) did you find most useful this week? Why?

2. **Sort Decision:** How do you decide whether to use `sort()` or `sorted()` in a real program?

3. **List Comprehensions:** When is a list comprehension better than a regular loop? When should you avoid it?

10 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

| My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

11 Bonus Section: Extra Challenges (5 extra points)

Only attempt if you've finished everything else!

11.1 Challenge 1: Binary Search Introduction (3 points)

Linear search checks every item one by one. But if a list is SORTED, we can search much faster using binary search. Research how binary search works and write a paragraph explaining the key idea:

How is it faster than linear search?

11.2 Challenge 2: Nested List Comprehension (2 points)

Create a list comprehension that generates this 2D pattern:

```
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

(Each row is the row number multiplied by 1, 2, and 3)

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-9)
- ☐ Saved each code exercise in its own file:
 - `week09_ex2.1.py` (find all positions)
 - `week09_ex2.2.py` (find first passing)

- `week09_ex5_1.py` (smallest even position)
 - `week09_ex6_fixed.py` (debugged code)
 - `cricket_analyzer.py` (full program)
- ☐ Created a folder named `week09_hw_yourname` with all files
 - ☐ Tested each code file to ensure it runs without errors
 - ☐ Drawn memory diagrams for Exercise 3.1
 - ☐ Filled in all trace tables
 - ☐ Verified search algorithms return correct results
 - ☐ Tested sorting with different lists
 - ☐ Checked list comprehensions produce expected output
 - ☐ Written your name at the top of this paper
 - ☐ Answered all reflection questions
 - ☐ Attempted bonus section (optional)

Which concept was most challenging? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job exploring list algorithms!

Next week: 2D Lists and Matrix Operations - lists within lists!